INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Course Title: Mobile Application Development

Assignment 2

Code Snippets and Explanation

SentimentAnalyzer (Android, Kotlin, TensorFlow Lite)

Submitted By

Fatima Tehreem (242201073)

BSCS-5B

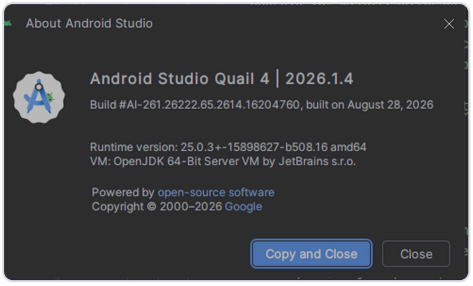
Course Instructor:

Sir Uzair Hassan

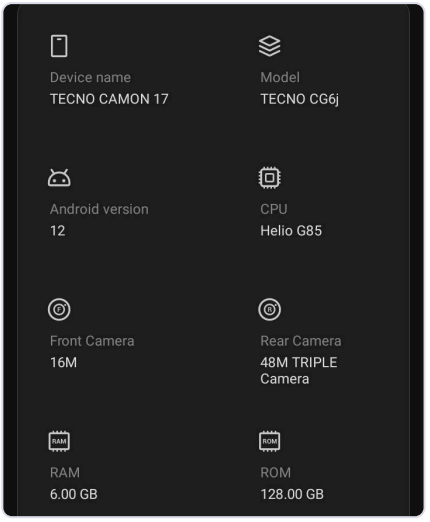
1. Project at a glance

SentimentAnalyzer is an Android app that takes a sentence typed by the user and uses an on-device TensorFlow Lite model to decide whether the sentiment is positive or negative. The model runs fully on the phone through the TensorFlow Lite Task Library's **NLClassifier**.

Item	Detail (from project screenshots)
Package	com.example.sentimentanalyzer
Main files	MainActivity.kt · activity_main.xml · app/build.gradle.kts
Model file	sentiment_model.tflite (751 KB) in app/src/main/assets
TFLite library	org.tensorflow:tensorflow-lite-task-text:0.4.4
IDE	Android Studio Quail 4 | 2026.1.4
Tested on	TECNO CAMON 17, Android 12



Android Studio version



Test device (My phone)

Note: all code below is taken from the project screenshots. Comments added for explanation are marked with //. Where a snippet says “not visible in the screenshot”, those exact lines were cut off in the supplied screenshots, so nothing has been guessed.

How the app works

1 · Load Model read from assets in onCreate › **2 · Input** Text read, trimmed, validated › **3 · Inference** classify(text) returns labels and scores › **4 · Output** Emoji, title, colour, confidence › **5 · Cleanup** Classifier closed in onDestroy

2. app/build.gradle.kts

This file configures the Android module and adds the TensorFlow Lite dependency that makes **NLClassifier** available to the code.

app/build.gradle.kts (complete file)

```
plugins {
    alias(libs.plugins.android.application)
}
```

```
android {
    namespace = "com.example.sentimentanalyzer"   // package name used by R and MainActivity

    compileSdk {
        version = release(37)
    }

    defaultConfig {
        applicationId = "com.example.sentimentanalyzer"
        minSdk = 24        // oldest supported Android version
        targetSdk = 37
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner =
            "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            optimization {
                enable = false
            }
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation(libs.androidx.activity.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.androidx.constraintlayout)
    implementation(libs.androidx.core.ktx)
    implementation(libs.material)

    // TensorFlow Lite text classification
    // [Note] provides NLClassifier used in MainActivity.kt
    implementation("org.tensorflow:tensorflow-lite-task-text:0.4.4")

    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)
}
```

gradle/libs.versions.toml (visible part)

```
[versions]
agp = "9.4.1"
coreKtx = "1.19.1"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
appcompat = "1.8.0"
material = "1.14.0"
activityKtx = "1.13.0"
constraintlayout = "2.2.2"

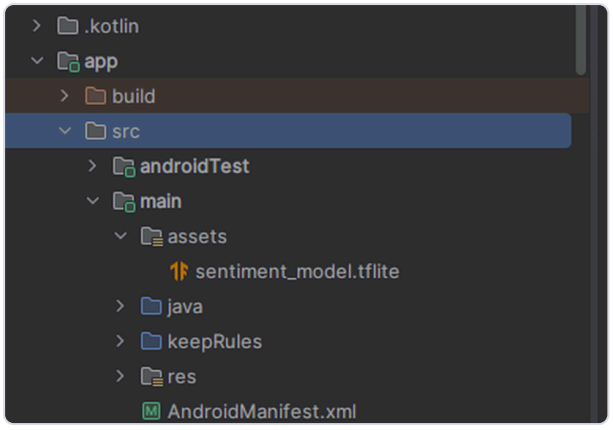
# [libraries] entries map the libs.* aliases used in build.gradle.kts
# (lines cut off on the right in the screenshot)

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
```

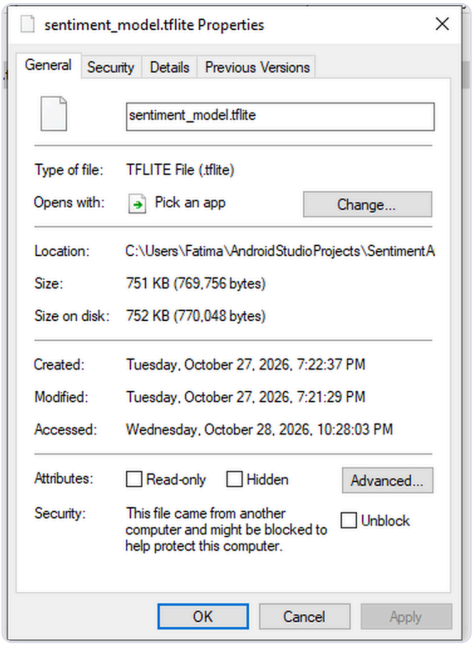
Explanation. `compileSdk` and `targetSdk` are 37 and `minSdk` is 24, so the app runs on Android 7.0 and above. The line `tensorflow-lite-task-text:0.4.4` is the only TensorFlow Lite dependency; it brings in the text-classification API. The other `libs.*` entries are AndroidX and Material libraries whose versions are defined in `libs.versions.toml`.

3. TensorFlow Lite model integration

Three things connect the model to the app: the model file placed in the **assets** folder, the **Gradle dependency** shown above, and the **NLClassifier** object created in MainActivity.kt.



Project tree: sentiment_model.tflite inside app/src/main/assets



File properties: TFLITE File, 751 KB (769,756 bytes)

Component	Role
sentiment_model.tflite	The trained sentiment model, packaged inside the APK from the assets folder.
tensorflow-lite-task-text	Library that provides the ready-made text classifier API.
NLClassifier	Loads the model, runs it on a String and returns a list of labels with scores.

NLClassifier at each stage

Stage	NLClassifier / code used	What happens
Model loading	`NLClassifier.createFromFile(this, "sentiment_model.tflite")`	Reads the model from assets once, in `onCreate()`.
Input processing	`text.trim()`, `isEmpty()`, null check	Cleans and validates the sentence before it reaches the model.
Inference	`model.classify(text)`	Returns labels with scores; the highest score is chosen.
Output display	`best.label`, `best.score`, `when { }`	Shows emoji, title, colour and confidence %.
Cleanup	`classifier?.close()`	Frees the classifier in `onDestroy()`.

4. MainActivity.kt

4.1 Class setup and views

MainActivity.kt: declarations

```
package com.example.sentimentanalyzer

// (other imports omitted)
import org.tensorflow.lite.task.text.nlclassifier.NLClassifier

class MainActivity : AppCompatActivity() {

    // UI components
    private lateinit var inputText: EditText
    private lateinit var analyzeButton: Button
    private lateinit var resultText: TextView
    private lateinit var resultEmoji: TextView
    private lateinit var resultDescription: TextView

    // TFLite classifier - nullable, so the app survives a failed model load
    private var classifier: NLClassifier? = null
```

MainActivity.kt: onCreate() view binding

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)   // inflate activity_main.xml

    // Connect Kotlin variables to the XML views by id
    inputText = findViewById(R.id.inputText)
    analyzeButton = findViewById(R.id.analyzeButton)
    resultText = findViewById(R.id.resultText)
    resultEmoji = findViewById(R.id.resultEmoji)
    resultDescription = findViewById(R.id.resultDescription)
```

Explanation. The five `lateinit` views are filled in `onCreate()` using their XML ids. The classifier is declared nullable (`NLClassifier?`) and starts as `null`, so the rest of the code must check it before use.

4.2 Model loading

MainActivity.kt: model loading (inside onCreate)

```
    try {
        // Load sentiment_model.tflite from app/src/main/assets
        classifier = NLClassifier.createFromFile(
            this,
            "sentiment_model.tflite"
        )
    } catch (e: Exception) {
        // Model missing or invalid: tell the user instead of crashing
        Toast.makeText(
            this,
            "Model could not load: ${e.localizedMessage}",
            Toast.LENGTH_LONG
        ).show()
    }
```

Explanation. `NLClassifier.createFromFile()` opens `sentiment_model.tflite` from the assets folder and prepares it for use. It runs once when the screen opens. If the file is missing or invalid, the `catch` block shows a Toast with the reason and `classifier` stays `null`; the app does not crash.

4.3 Input processing

MainActivity.kt: button click, input checks

```
    analyzeButton.setOnClickListener {
        // Read the sentence and remove leading/trailing spaces
```

```
        val text = inputText.text.toString().trim()

        // Validation: empty input is rejected
        if (text.isEmpty()) {
            inputText.error = "Please enter some text"
            return@setOnClickListener
        }

        // Safety check: model must have been loaded in onCreate
        val model = classifier

        if (model == null) {
            resultEmoji.text = "⚠️"
            resultText.text = "Model not loaded"
            resultDescription.text =
                "Check the TensorFlow Lite model in app/src/main/assets."
            return@setOnClickListener
        }
```

Explanation. When the user taps *Analyze Sentiment*, the text is read from the EditText and `trim()` removes extra spaces. Empty input stops with an error message on the field. The code then checks that the model really loaded. The XML limits input to 500 characters (`maxLength`). Converting the sentence into the numeric form the model needs is handled inside `NLClassifier`, so the app passes the plain String.

4.4 Inference

MainActivity.kt: running the model

```
        try {
            // INFERENCE: the model scores the sentence for each label
            val results = model.classify(text)

            if (results.isEmpty()) {
                resultEmoji.text = "❓"
                resultText.text = "No prediction"
                resultDescription.text = "The model returned no results."
                return@setOnClickListener
            }

            // Pick the label with the highest score
            val best = results.maxByOrNull { it.score }

            if (best == null) {
                resultEmoji.text = "❓"
                resultText.text = "No prediction"
                resultDescription.text = "Please try another sentence."
                return@setOnClickListener
            }
```

Explanation. `model.classify(text)` is the inference step. It returns a list of results, each with a `label` and a `score`. `maxByOrNull { it.score }` selects the most confident result. Two guards handle an empty list and a missing best result, each showing a “No prediction” message.

4.5 Output display

MainActivity.kt: showing the result

```
            // Normalise the label and convert the score to a percentage
            val label = best.label.trim().lowercase()
            val confidence = (best.score * 100).toInt()

            // Map the label to an emoji, a title and a colour
            when {
                label.contains("positive") ||
                        label == "pos" ||
                        label == "1" -> {
                    resultEmoji.text = "😊"
                    resultText.text = "Positive Sentiment"
                    resultText.setTextColor(
                        Color.rgb(22, 143, 120)
                    )
```

```
                }

                label.contains("negative") ||
                        label == "neg" ||
                        label == "0" -> {
                    resultEmoji.text = "😔"
                    resultText.text = "Negative Sentiment"
                    resultText.setTextColor(
                        Color.rgb(205, 75, 100)
                    )
                }

                else -> {
                    resultEmoji.text = "🔎"
                    resultText.text = "Model Result"
                    resultText.setTextColor(
                        Color.rgb(80, 75, 150)
                    )
                }
            }

            // Show the raw label and the confidence
            resultDescription.text =
                "Model label: ${best.label}\nAI confidence: $confidence%"

        } catch (e: Exception) {
            resultEmoji.text = "⚠️"
            resultText.text = "Prediction failed"
            resultDescription.text =
                e.localizedMessage ?: "Please check the model file."
        }
    }
}
```

Explanation. The label is lower-cased and the score is turned into a whole-number percentage. A when block then chooses what to show: a positive label gives 😊, “Positive Sentiment” and a green colour; a negative label gives 😔, “Negative Sentiment” and a red colour; anything else shows 🔎 and “Model Result”. The description line prints the model label and the confidence. If anything fails, the outer `catch` shows “Prediction failed”.

4.6 Releasing the model

MainActivity.kt: onDestroy()

```
override fun onDestroy() {
    classifier?.close()   // release native TFLite resources
    classifier = null
    super.onDestroy()
}
```

Explanation. `close()` frees the native memory held by the TensorFlow Lite classifier when the activity is destroyed.

5. activity_main.xml

The layout is a `ScrollView` containing a vertical `LinearLayout` with a title, an input card and a result card. The ids that Kotlin looks up with `findViewById` (`analyzeButton`, `resultEmoji`, `resultText`, `resultDescription`) are defined here, along with the input field.

activity_main.xml: root layout and title

```
<?xml version="1.0" encoding="utf-8"?>
<!-- ScrollView: whole screen can scroll on small phones -->
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="#F2F3FF">

    <!-- Vertical container for title, input card and result card -->
```

```
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="28dp"
            android:gravity="center"
            android:text="🧠 AI Sentiment Analyzer"
            android:textColor="#25245C"
            android:textSize="28sp"
            android:textStyle="bold" />
```

activity_main.xml: input card (EditText and Button)

```
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="18dp"
            android:background="@drawable/card_white">

            <!-- Input field read in MainActivity (R.id.inputText); maxLength limits it to 500 characters -->
            <EditText
                [lines 51-52 not visible in the screenshot]
                android:layout_height="150dp"
                android:background="@drawable/input_background"
                android:gravity="top|start"
                android:hint="Write a review or comment here..."
                android:inputType="textCapSentences|textMultiLine"
                android:maxLength="500"
                android:padding="14dp"
                android:textColor="#292750"
                android:textColorHint="#9998B5"
                android:textSize="16sp" />

            <!-- Button that triggers setOnClickListener in MainActivity -->
            <Button
                android:id="@+id/analyzeButton"
                android:layout_width="match_parent"
                android:layout_height="58dp"
                android:layout_marginTop="20dp"
                android:backgroundTint="#7775E8"
                android:clickable="true"
                android:enabled="true"
                android:focusable="true"
                android:text="Analyze Sentiment"
                android:textAllCaps="false"
                [lines 75-76 not visible in the screenshot]
                android:textStyle="bold" />
        </LinearLayout>
```

activity_main.xml: result card

```
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="22dp"
            android:background="@drawable/result_background"
            android:gravity="center"
            android:orientation="vertical"
            android:padding="22dp">

            <!-- Emoji updated by Kotlin (resultEmoji.text) -->
            <TextView
                android:id="@+id/resultEmoji"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="18dp"
                android:text="💭"
                android:textSize="46sp" />

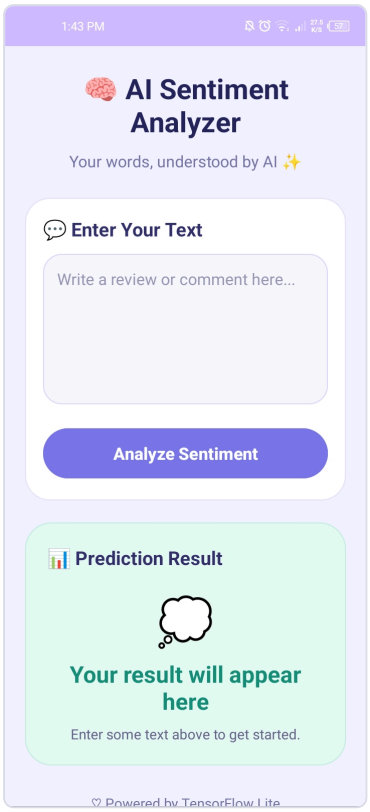
            <!-- Sentiment title and colour set by Kotlin -->
            <TextView
                android:id="@+id/resultText"
```

```
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:gravity="center"
            android:text="Your result will appear here"
            android:textColor="#168F78"
            android:textSize="23sp"
            android:textStyle="bold" />

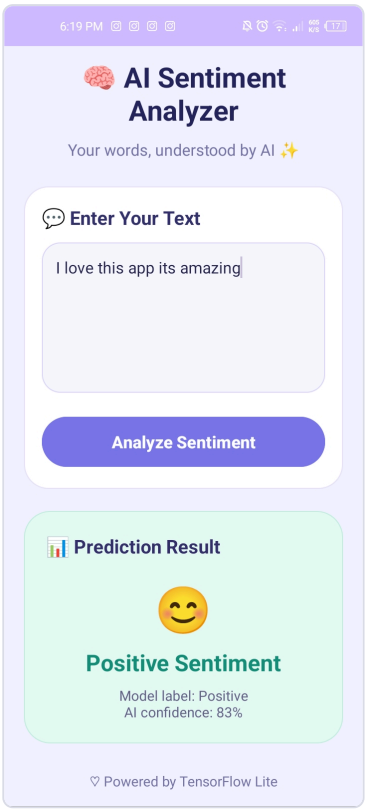
        <!-- Model label and confidence line -->
        <TextView
            android:id="@+id/resultDescription"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:gravity="center"
            android:text="Enter some text above to get started."
            android:textColor="#64648A"
            android:textSize="14sp" />
    </LinearLayout>
```

Explanation. The result card starts with default text (“Your result will appear here”) and a 💭 emoji. After analysis, MainActivity.kt replaces these three TextViews. The drawables `card_white`, `input_background` and `result_background` give the rounded card look.

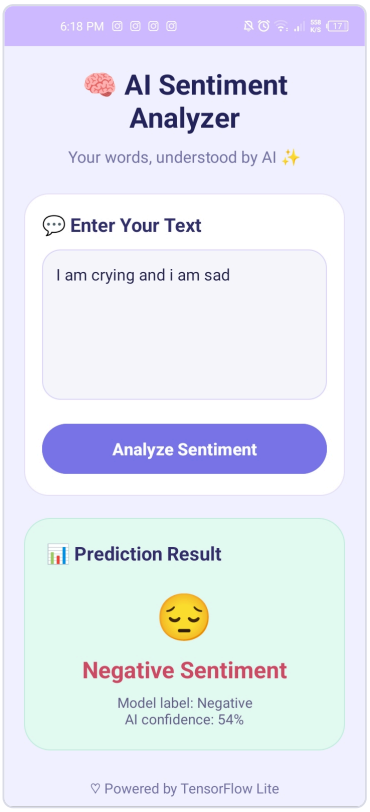
6. App output



Initial screen (default result card)



Positive result: label and 83% confidence



Negative result: label and 54% confidence

Each screen above corresponds to a branch in section 4: the default layout values (section 5), the positive branch and the negative branch of the when block.